{EPITECH}

CONNECT'IN_

< DÉVELOPPEMENT D'UN RÉSEAU SOCIAL INTERNE POUR UNE ESN />



CONNECT'IN

Contexte professionnel

Vous êtes **développeur junior au sein d'une ESN (Entreprise de Services du Numérique)** comptant plusieurs équipes réparties sur différents projets et sites.

Afin de **renforcer le lien entre les collaborateurs**, de favoriser les échanges informels et de créer un sentiment d'appartenance, la direction souhaite mettre en place un **réseau social interne** réservé aux salariés de l'entreprise.

Vous êtes chargé de concevoir et développer **Connect'In**, une application web permettant aux collaborateurs :
- de publier des messages,
- de commenter et réagir aux publications,
- d'interagir avec leurs collègues dans un cadre professionnel et sécurisé.

Ce réseau social est **strictement interne à l'entreprise** et nécessite une authentification.

Présentation du projet

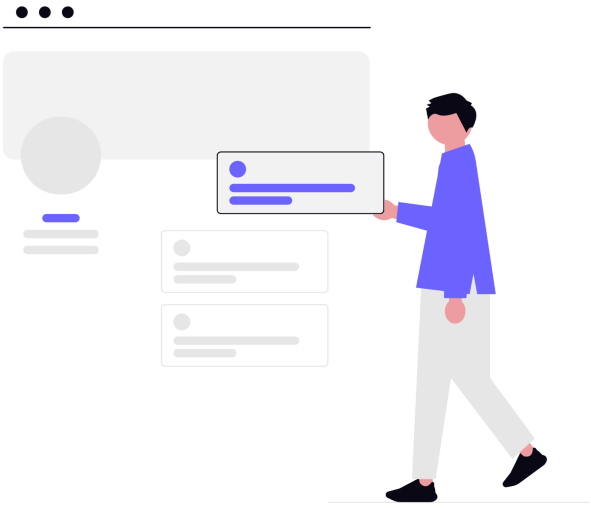
Le projet consiste à développer un **mini réseau social interne**, basé sur une **architecture moderne séparant clairement le back-end et le front-end**.

- ✓ Le **back-end** est une **API RESTful** développée avec **Laravel (PHP, POO, MVC)** et **documentée**.
- ✓ Le **front-end** est une application web qui **consomme l'API** via JavaScript (`fetch` ou Axios par exemple).
- ✓ Les données sont stockées dans une base **MySQL**.
- ✓ L'interface est conçue avec **Tailwind CSS**, responsive et accessible.



L'utilisation d'un framework JavaScript (React, Angular, Vue.js, etc) n'est pas demandée, mais n'est pas interdite pour autant.

Les utilisateurs pourront :
- créer un compte et se connecter,
- publier, modifier et supprimer des posts,
- commenter et liker les publications,
- gérer leur profil.





- ✓ Toutes les actions doivent être **sécurisées** (validation des données, authentification, autorisations).
- ✓ Le back-end **ne doit pas générer de vues HTML** : il expose uniquement des endpoints API.
- ✓ Le front-end consomme exclusivement l'API pour toutes les interactions.
- ✓ L'application doit respecter les bonnes pratiques de Laravel et de la programmation orientée objet.

Objectifs pédagogiques

L'objectif principal est de concevoir un **réseau social interne fonctionnel, sécurisé et évolutif**.

Ce projet permet de travailler les compétences suivantes :

- ✓ Conception d'une **architecture API REST** avec Laravel.
- ✓ Gestion de l'**authentification via API** (utilisation de tokens).
- ✓ Modélisation et gestion d'une **base de données relationnelle MySQL**.
- ✓ Création et utilisation de **migrations, modèles et relations Eloquent**.
- ✓ Consommation d'une API via **JavaScript (fetch / Axios)**.
- ✓ Création d'une interface moderne et responsive avec **Tailwind CSS**.
- ✓ Travail en **équipe**, organisation et documentation.

Fonctionnalités attendues

Gestion des utilisateurs

Les utilisateurs doivent pouvoir s'inscrire (à minima email professionnel, prénom, nom, mot de passe), se connecter et se déconnecter.

Les utilisteurs non connectés ne peuvent que consulter la page de connexion et d'inscription.

Les utilisateurs connectés doivent pouvoir accéder à une page de profil pour accéder et modifier leurs informations (et mot de passe). Ils doivent également pouvoir supprimer leur compte.

Note : lorsqu'un utilisateur supprime son compte, il doit pouvoir :
- choisir de supprimer le contenu associé (commentaires, posts)
- choisir de laisser son contenu. Dans ce cas, son nom/prénom sur les posts et commentaires est remplacé par ”Utilisateur supprimé”.

Dans tous les cas, l'utilisateur en lui-même est supprimé, ainsi que l'ensemble de ses likes.

Gestion des posts

Chaque utilisateur connecté peut créer un post (avec au moins texte et image optionnelle), modifier et supprimer **ses propres posts**.

Tous les utilisateurs peuvent consulter les posts de tout le monde. Ils peuvent également liker ou retirer un like sur un post (un seul like par utilisateur et par post).

Note : en cas de suppression d'un post, tous les commentaires associés doivent être supprimés.

Gestion des commentaires

Chaque post peut avoir plusieurs commentaires. Les utilisateurs peuvent ajouter un commentaire sous un post, et modifier ou supprimer **leurs propres commentaires**.

Chaque action réalisée sur le site une fois connecté (ajout d'un post, d'un like, d'un commentaire) doit être dynamique ; il ne doit pas être nécessaire de recharger la page pour voir son post/commentaire/like.

Conception de la base de données

Pour ce projet, vous devez concevoir votre base de données, aucune modélisation ne vous est fournie.

Un **diagramme de base de données** est attendu dans les livrables. Veuillez à ce que votre conception réponde bien à l'ensemble des fonctionnalités décrites ci-dessus.

Étapes de réalisation recommandées

Ce projet est un projet long, à réaliser **en équipe**.

Il est fortement conseillé de suivre les étapes suivantes :

1. Analyse du besoin et lecture complète du sujet
2. Définition des fonctionnalités et des règles métier
3. Conception de la base de données (MCD / schéma relationnel)
4. Création des entités Laravel
5. Mise en place de l'API :
   - ✓ routes
   - ✓ contrôleurs
   - ✓ modèles
6. Implémentation de l'authentification API
7. Développement progressif des fonctionnalités :
   - ✓ utilisateurs
   - ✓ posts
   - ✓ commentaires
   - ✓ likes
8. Développement du front-end
9. Tests manuels complets de votre projet :
   - ✓ cas normaux
   - ✓ erreurs
   - ✓ sécurité
10. Documentation de l'API et rédaction du README

Bonus (facultatif)

Pour aller plus loin, et une fois que toutes les fonctionnalités attendues ont été réalisées, vous pouvez ajouer des fonctions supplémentaire à votre réseau social. Par exemple :
- Photo de profil utilisateur
- Groupes ou équipes internes
- Filtrage des posts (date, popularité)
- Système de notifications
- Pagination des posts

- Recherche de publications
- Système de rôles et modération

Livrables attendus

- ✓ Code source complet du projet
- ✓ README :
  - installation
  - configuration
  - utilisation de l'API
- ✓ Diagramme de la base de données
- ✓ Documentation de l'API (Postman, Swagger, ...)

Ressources utiles

- ✓ Laravel — https://laravel.com/docs
- ✓ Tailwind CSS — https://tailwindcss.com/docs
- ✓ MySQL — https://www.mysql.com
- ✓ Axios — https://axios-http.com/docs/intro
- ✓ MDN Web Docs — https://developer.mozilla.org

v 1.0.0

